```
app.UseAuthorization();                          ❺

app.MapGet("/recipe", async (RecipeService service) =>
{
    return await service.GetRecipes();
}).RequireAuthorization();                       ❻

app.Run();
```

❶ Adds the core authentication services

❷ Adds and configures JWT authentication

❸ Adds the core authorization services

❹ Adds the authentication middleware

❺ Adds the authorization middleware

❻ Adds an authorization policy to the minimal API endpoint

As well as configuring the JWT authentication, listing 25.1 adds an authorization policy to the one minimal API endpoint shown in the app. The `RequireAuthorization()` function adds a simple "Is authenticated" authorization policy to the endpoint. This is exactly analgous to when you add an `[Authorize]` attribute to MVC or Web API controllers. Any requests for this endpoint must be authenticated; otherwise, the request is rejected by the authorization middleware with a `401 Unauthorized` reponse, as shown in figure 25.9.

Authentication schemes: Choosing between cookies and bearer tokens

One question you may have while reading about bearer authentication is how the authentication middleware knows whether to look for the cookie or a header. The answer is authentication schemes.

An *authentication scheme* in ASP.NET Core has an ID and an associated *authentication handler* that controls how the user is authenticated, as well as how authentication and